

Practice 1

Time: 15 minutes

You are building the connectivity layer for a data-processing application. The application opens a connection to a shared resource pool used to fetch records from a remote service. Opening this pool is expensive (it negotiates credentials and reserves memory), and if two different parts of the program each open their own pool, the remote service starts rejecting requests for exceeding the allowed number of concurrent pools.

The general requirement: 1. The pool must be set up the first time some part of the program actually needs it. 2. Every part of the program that asks for the pool afterwards must be handed that same, already-set-up pool. 3. Nothing outside the pool class should be able to create a second pool object by any means.

Task: Implement the connectivity layer so that the resource pool satisfies the requirement above.

- The pool object must not be constructible from outside its own class.
- Two separate parts of the program that request the pool must end up referring to the exact same object.
- The pool should not be created until it is first requested.

Practice 2

Time: 20 minutes

You are developing an online checkout module. The store accepts payments through Credit Card, PayPal, and Crypto Wallet. All payment methods implement a common interface **Payment** with methods **validate()** and **charge()**.

The general checkout steps are the same: 1. Create the payment object for the chosen method. 2. Validate the payment details. 3. Charge the customer. 4. Print a receipt confirmation message.

However, different checkout handlers should decide which concrete payment object is created, depending on the method the handler is responsible for.

Task: Implement the module so that the common checkout steps are written once, while specialized handler classes decide the concrete payment type to use.

- The class containing the common checkout steps must not directly create Credit Card, PayPal, or Crypto Wallet payment objects.
- Adding a new payment method later should not require editing the common checkout steps.
- Simple print statements are sufficient inside **validate()** and **charge()**.

Practice 3

Time: 25 minutes

You are building a furniture catalog generator for a manufacturer. The manufacturer produces two complete furniture lines: Modern and Victorian. Each line offers a matching **Chair** and a matching **Sofa**. A showroom set may never mix a Modern chair with a Victorian sofa, or vice versa — the two pieces shown together must always come from the same line.

The general requirement: 1. A showroom-set builder asks for one chair and one sofa without stating "Modern" or "Victorian" directly in its own logic. 2. Whichever line was selected earlier in the program determines both pieces that get produced. 3. It must be structurally impossible to hand back a chair from one line paired with a sofa from the other.

Task: Implement the catalog generator so that a showroom set can be produced for either line without the set-building logic ever naming a concrete chair or sofa class.

- Adding a third furniture line (e.g. Industrial) later should not require changes to the code that assembles a showroom set.
- Simple print statements are sufficient inside chair/sofa methods.

Practice 4

Time: 20 minutes

You are developing an order system for a sandwich shop. A **Sandwich** order has a bread type and a filling (both required), and may optionally include cheese, sauce, and vegetables in any combination the customer wants. Handling every possible combination of optional items through one constructor with many parameters has already caused bugs where two optional items were accidentally swapped by the kitchen staff reading the order.

The general requirement: 1. An order is put together one item at a time — bread and filling first, then whichever optional items the customer asked for. 2. The shop also offers two fixed combos, "Classic Combo" and "Spicy Combo", that should be assembled using the exact same step-by-step process as a custom order, without duplicating the assembly steps for each combo. 3. Once an order is handed to the kitchen, it must not be possible to change any of its items.

Task: Implement the order system so that both custom orders and the two fixed combos are produced through the same reusable step-by-step assembly process, and the finished order object cannot be modified afterward.

- Simple print statements are sufficient for each assembly step.